Compte rendu de réunion n°2 : TER BYZFL

Approfondissement théorique et planification des benchmarks ByzFL

ABBES Samy & AREZKI Célia

11 Mai 2026

1 Objectifs et Prise en main

Suite à la validation de notre premier script fonctionnel sur le jeu de données MNIST (IID) étalé sur 1000 rounds, cette deuxième réunion de travail a permis de définir les axes théoriques à creuser et de structurer la suite de nos campagnes d'expérimentations. L'objectif principal donné par notre encadrant est de passer d'une simple exécution de scripts à une compréhension fine des mécanismes mathématiques sous-jacents, tout en industrialisant la collecte de nos résultats.

2 Directives théoriques et Concepts à expliciter

2.1 Formulations mathématiques des attaques et agrégateurs

Il a été souligné l'importance de maîtriser les équations régissant la falsification des gradients par les clients malveillants. Les attaques ne génèrent pas un bruit aléatoire, mais visent un objectif précis via des formules mathématiques, par exemple :

— **Inner Product Manipulation (IPM) :** Repose sur la manipulation du produit scalaire. L'attaquant calcule la moyenne des gradients honnêtes (μ_h) et envoie un gradient opposé multiplié par un facteur d'échelle τ, tel que :

$$g_{byz} = -\tau \cdot \mu_h$$

Le but est de forcer le produit scalaire global à devenir négatif, inversant ainsi la trajectoire d'apprentissage.

— **A Little Is Enough (ALIE) :** Utilise la moyenne (μ_h) et l'écart-type (σ_h) des clients sains pour introduire une perturbation. L'équation de l'attaque est :

$$g_{byz} = \mu_h - \tau \cdot \sigma_h$$

Le paramètre τ définit le nombre d'écarts-types dont l'attaquant s'éloigne. Bien calibré, il permet de biaiser le modèle tout en restant indétectable par les filtres statistiques du serveur.

2.2 Mécanisme de Krum et sélection géométrique

L'agrégateur *Krum* évalue la distance euclidienne de chaque gradient vis-à-vis de ses voisins. Pour attribuer un score à un client, il effectue la somme des distances de ses $n - f - 2$ voisins les plus proches. Cette formule s'explique par la nécessité mathématique d'exclure les f gradients potentiellement byzantins les plus éloignés, le gradient du client lui-même (1), ainsi qu'une marge de sécurité unitaire (1) pour garantir la dominance des nœuds honnêtes lors du calcul du score de proximité.

2.3 La limite théorique du tiers byzantin (1/3)

Un point d'attention crucial a été soulevé concernant la borne supérieure de tolérance aux pannes byzantines. Lorsque le nombre d'adversaires atteint ou dépasse le seuil critique de $f \geq n/3$, il devient mathématiquement impossible de distinguer un groupe de clients honnêtes d'un groupe d'attaquants coordonnés. À ce stade, les garanties de convergence des agrégateurs géométriques s'effondrent.

2.4 Rôle du Momentum

Le *momentum* agit comme une mémoire pour l'optimiseur. Au lieu de réagir uniquement au gradient du round actuel, le système conserve une trace des directions précédentes (effet « boule de neige »). En réduisant ce paramètre, on limite volontairement cette inertie. Ainsi, si un attaquant byzantin réussit à injecter une mauvaise direction, celle-ci sera très vite oubliée par le modèle et ne continuera pas à polluer les rounds suivants.

3 Protocole expérimental et Plan d'action

3.1 Définition des scénarios de Robustesse

Afin de mettre en évidence l'intérêt des défenses de la bibliothèque `ByzFL`, nous devons implémenter trois configurations distinctes pour chaque couple attaque/défense :

1. **Système sain sans défense :** Utilisation de la moyenne simple (*Average*) sans adversaires byzantins (Baseline idéale).
2. **Système attaqué sans défense :** Utilisation de la moyenne simple confrontée à des attaques (*SignFlipping, IPM, ALIE*). Ce scénario mettra en évidence la vulnérabilité par défaut du système.
3. **Système attaqué avec défense :** Confrontation directe des agrégateurs robustes (*Median, TrMean, Krum*) face aux attaques byzantines.

3.2 Exploration des hyperparamètres

Nous devons faire varier de manière systématique :

— Le nombre d'itérations d'apprentissage (`NB_ROUNDS`).

— La taille des `BATCH` chez les clients pour analyser l'impact du bruit de quantification sur la détection des anomalies.

3.3 Industrialisation et Analyse Graphique

Pour exploiter ces données, nous allons encapsuler notre pipeline dans des fonctions génériques paramétrables. Les résultats d'accuracy (initiale et finale) ainsi que les pertes seront exportés automatiquement dans des fichiers au format `.csv`. Ces fichiers serviront pour le traçage de courbes de convergence comparatives.

4 Aspects techniques et Environnement

4.1 Accélération matérielle (GPU)

Afin de mener à bien nos différentes campagnes d'expérimentations impliquant de multiples combinaisons et un grand nombre d'itérations et donc nécéssitant beaucoup de puissance de calcul, nous envisageons l'utilisation de l'accélération matérielle via GPU. Cela nous permettra de réduire significativement les temps de calcul lors de l'entraînement du modèle fédéré.

4.2 Gestion de l'environnement avec UV

Pour garantir la portabilité du code et la reproductibilité parfaite des benchmarks entre les membres de l'équipe et l'encadrant, nous abandonnons la gestion classique par `pip` au profit de l'outil `uv`. Un fichier de configuration sera couplé à `uv venv` pour figer l'environnement virtuel d'exécution.

5 Conclusion et Prochaines étapes

L'objectif d'ici la prochaine réunion sera de mettre en pratique l'ensemble des directives et conseils établis lors de ces échanges. Il s'agira concrètement de configurer notre nouvel environnement de travail avec les bons outils, de faire en sorte d'automatiser la récolte des données de benchmarking, et de lancer les différentes séries de tests couvrant tous les scénarios d'attaques et de défenses demandés.